promoterai-torch: a PyTorch port of Illumina's PromoterAI

Adam Youlin He, Anshul Kundaje

Stanford University

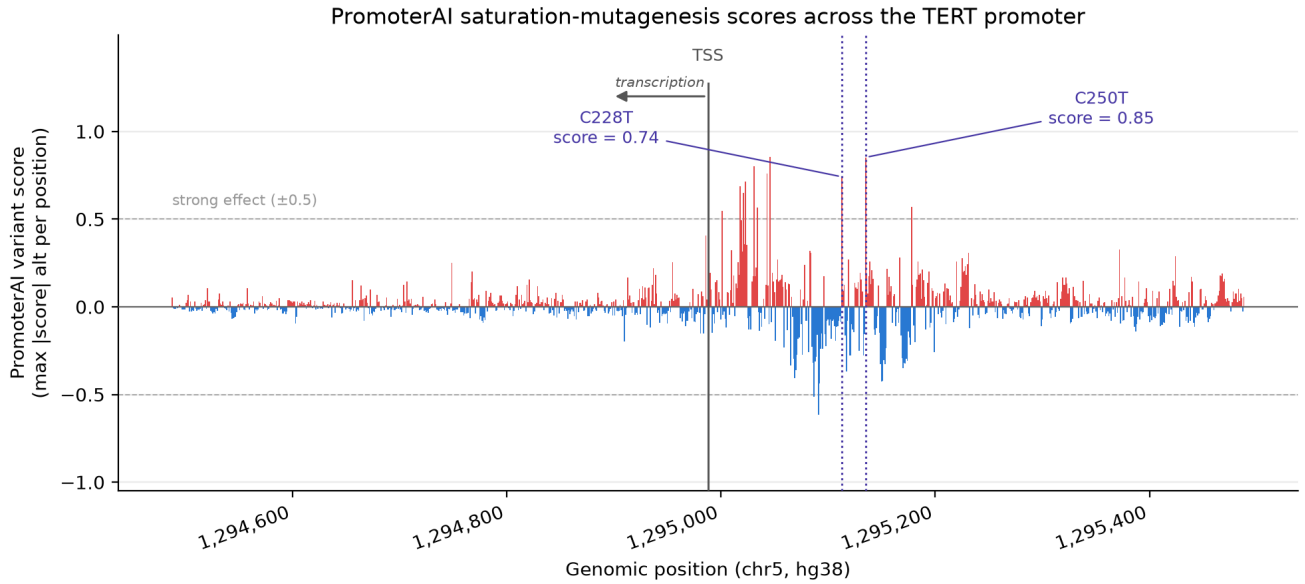
Genomics x AI Blog, 29 September 2026. https://doi.org/10.5281/zenodo.23038831
Online version: https://genomicsxai.github.io/blogs/2026-014/

Summary

PromoterAI [1] predicts how promoter variants alter gene expression, but the official release ships as a TensorFlow/Keras SavedModel [2]. `promoterai-torch` is an independent, numerically-equivalent PyTorch port that converts Illumina's checkpoints and makes variant scoring, track prediction, embedding extraction, and DeepLIFT/SHAP attribution available through the PyTorch ecosystem, with training and fine-tuning scripts included for anyone who wants to reproduce or extend the model from scratch.

Overview

Promoters are a critical class of non-coding regulatory DNA elements that set the baseline transcriptional output of a gene. A single nucleotide change just upstream of the transcription start site can create or destroy a transcription factor binding site without touching the coding sequence of a gene. The best-known example is *TERT*: two recurrent, mutually exclusive promoter mutations, independently identified in melanoma [4, 5] and subsequently found in glioblastoma, bladder cancer, and dozens of other tumor types [6], each create a de novo ETS/GABP transcription-factor binding motif upstream of the transcription start site, driving aberrant telomerase re-expression [4–7]. Both are in the *TERT* variant set checked below — C228T and C250T, chr5:1,295,113 G>A and chr5:1,295,135 G>A on the hg38 minus strand — and PromoterAI's ensembled scores for them are 0.74 and 0.85 respectively: both comfortably past the paper's ±0.5 "strong effect" threshold and positive, consistent with the gain-of-function reported in the literature.



PromoterAI saturation-mutagenesis scores across the TERT promoter (chr5, hg38); bar height is the largest-magnitude signed score among the 3 possible alt alleles at each position. TERT is on the minus strand, so transcription proceeds toward decreasing coordinate (arrow). The recurrent C228T and C250T mutations (chr5:1,295,113 G>A and chr5:1,295,135 G>A) both score well past the paper's +0.5 strong-effect threshold.

Prioritizing functional promoter variants more broadly is a hard, unsolved problem in variant interpretation. PromoterAI addressed this by training a sequence-to-function model on hundreds of regulatory tracks (histone marks, TF ChIP-seq, ATAC-seq, RNA-seq) across human and mouse promoters, then fine-tuning on expression outlier variants (with signed differences between reference and alternate predictions as a variant effect score).

However, the official release of PromoterAI is in TensorFlow/Keras, which has effectively walled PromoterAI off from the PyTorch-based S2F ecosystem since it shipped. Getting per-base attributions out of PromoterAI today means reimplementing DeepLIFT/SHAP's gradient-correction rules against `tf.GradientTape` — a substantial undertaking, and easy to get subtly wrong. Similarly, gradient-based sequence design with tools like `Ledidi` [8], which optimizes a sequence toward a target prediction by backpropagating through the model, needs the same kind of direct, differentiable access. Scoring the same variant set across PromoterAI and PyTorch-native models, to compare or ensemble their predictions, means round-tripping through disk rather than composing tensors directly. `promoterai-torch` re-implements the architecture layer-for-layer in PyTorch and ships a converter that reads an existing Illumina SavedModel and produces a `.pt` checkpoint, so PromoterAI becomes just another `nn.Module` that plugs into existing PyTorch pipelines — attribution, design, and ensembling with the rest of the PyTorch S2F ecosystem all become drop-in rather than bespoke.

Porting sequence-to-function models to PyTorch this way is a familiar, worthwhile pattern for the community rather than a one-off: `enformer-pytorch` ports Enformer [9], Flashzoi provides an accelerated PyTorch reimplementation of Borzoi [10], and we previously covered porting AlphaGenome to PyTorch on this blog [11]. `promoterai-torch` follows the same playbook for PromoterAI.

Architecture

PromoterAI's backbone is a "MetaFormer"-style stack: a 1×1 convolution stem projects the one-hot DNA sequence into a `model_dim`-wide channel space, followed by `num_blocks` residual blocks. Each block alternates two mixing operations behind BatchNorm and a residual connection — a depthwise, dilated 1D convolution across positions (token mixing, with the dilation rate held at 1 for the first four blocks, then doubling every two blocks thereafter: 1, 1, 1, 1, 2, 2, 4, 4, …) and a two-layer feed-forward network across channels (channel mixing). This is the same token-mixing/channel-mixing split used by "MetaFormer"-family vision architectures, with a convolutional token mixer in place of pooling or attention.

Prediction happens through per-species output heads that read out from several depths of the backbone rather than just the final block: every `shortcut_layer_freq`-th block's hidden state is linearly projected to that species' track dimension, passed through a ReLU, and the resulting projections are averaged and center-cropped to the model's output length. The released checkpoints carry two such heads sharing one backbone — a 498-track human head and a 472-track mouse head. Only the human head is unfrozen during PromoterAI's own variant-effect fine-tuning; the rest of the network, including BatchNorm running statistics, stays in eval mode throughout.

> This is **not** an official Illumina product or publication. `promoterai-torch`'s PyTorch code is an independent reimplementation, and its release should not be construed as endorsed by Illumina. The pretrained *weights* remain gated under Illumina's own license, independent of this port's own code license — see License below for the full picture, and please don't redistribute converted checkpoints.

Getting Started

The core package installs without pulling in TensorFlow, HDF5/BigWig tooling, or attribution libraries:

```
pip install promoterai-torch
```

Converting a pretrained checkpoint requires the `[convert]` extra and a copy of the official SavedModel from Illumina/PromoterAI. Note that Illumina gates the pretrained SavedModels and precomputed variant scores behind a signed academic-use license agreement (commercial licensing goes through `AI_licensing@illumina.com`) — see their README for the request form first.

```
pip install "promoterai-torch[convert]"

promoterai-torch convert \
    --keras_model models/promoterAI_v1_hg38_mm10_finetune \
    --output models/promoterAI_v1_hg38_mm10_finetune.pt \
    --input_length 20480 \
    --output_length 4096
```

Architecture hyperparameters (`num_blocks`, `model_dim`, `output_dims`) are inferred automatically from the SavedModel, so this works for any PromoterAI-architecture checkpoint — not just Illumina's four released checkpoints, but your own Keras fine-tunes produced by `promoterai.finetune` as well.

From there, scoring a variant TSV (`chrom`, `pos`, `ref`, `alt`, `strand`) is one command:

```
promoterai-torch score \
    --model_checkpoint models/promoterAI_v1_hg38_mm10_finetune.pt \
```

```
    --var_file variants.tsv \
    --fasta_file hg38.fa \
    --input_length 20480
```

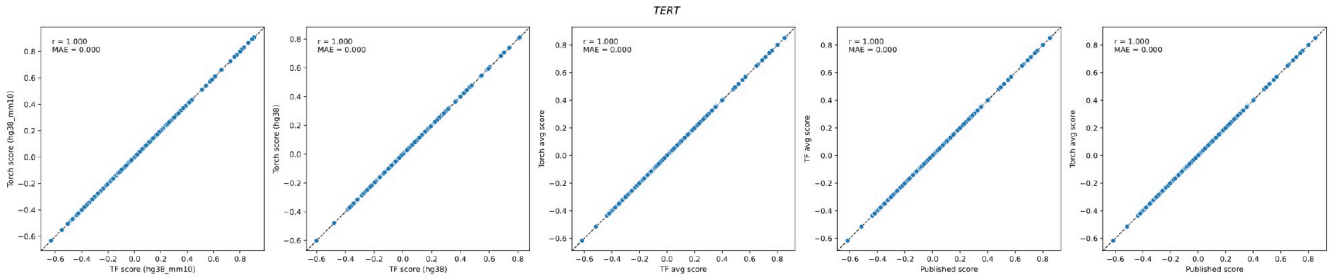
Scores land in [−1, 1], with the same effect-size thresholds as the original paper (±0.1 weak, ±0.2 moderate, ±0.5 strong).

Numerical Equivalence

Porting a model is only useful if it actually reproduces the original, so most of the engineering effort went here.

Track-level equivalence. Running both the original TF/Keras SavedModel and the converted PyTorch checkpoint on the same sequences and comparing every output track gives errors of ~1e-7 at FP32 — within machine precision — across all four released checkpoints (`hg38`, `hg38_mm10`, `hg38_finetune`, `hg38_mm10_finetune`).

Variant-score equivalence. On promoter variants at *TERT* (*n* = 6,006), *SFSWAP* (*n* = 3,003), and *DNAJC9* (*n* = 9,009), torch and TF/Keras variant scores are identical, including the ensembled score used in the paper and distributed in the official scores published by Illumina (Pearson *r* = 1.0000, MAE = 0.0000). Note that the scoring script/CLI in both the official repo and this port round the score to 4 digits, which is why variant scores will generally be identical.

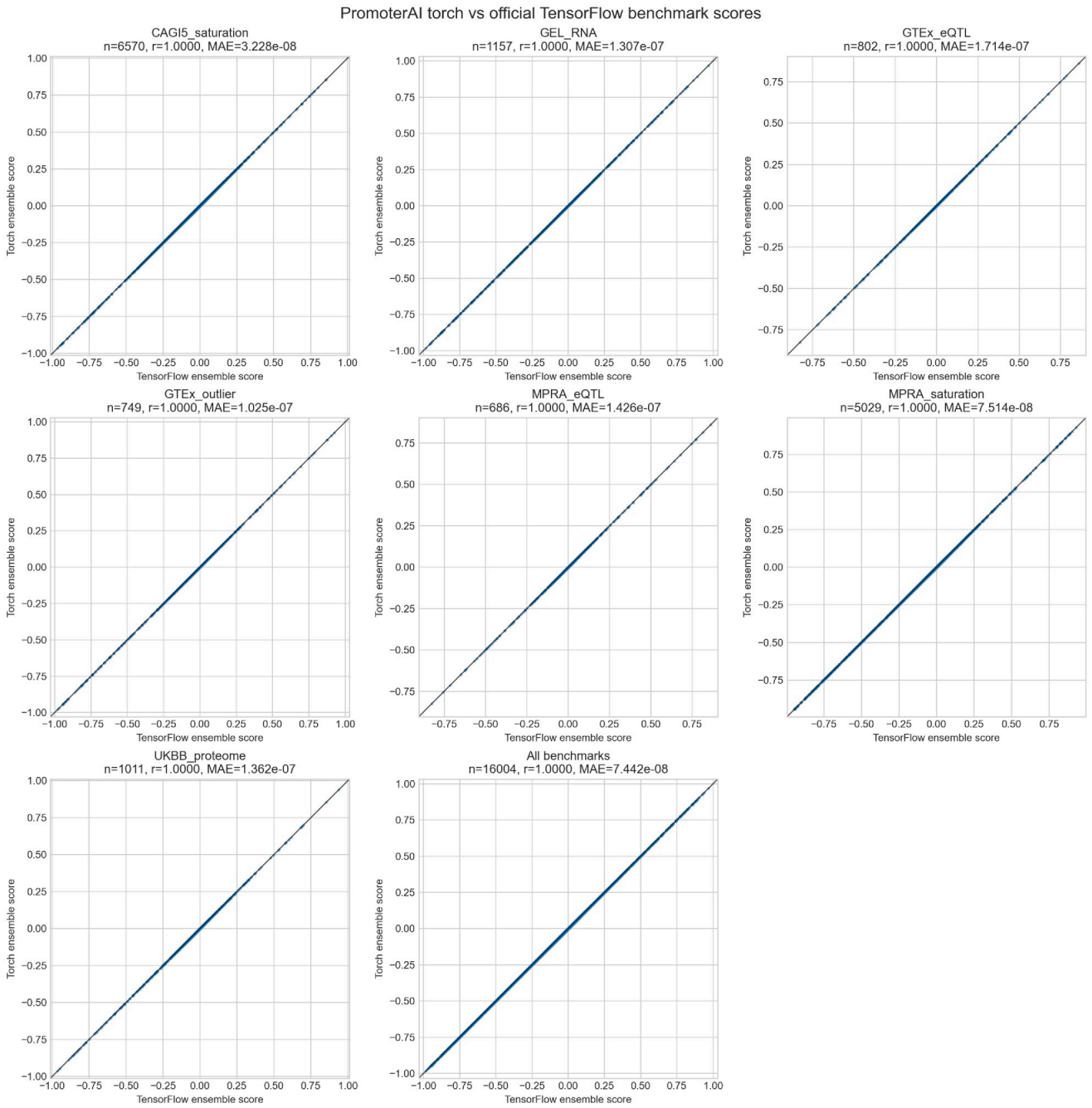


TERT promoter variant scores (n = 6,006): per-checkpoint and ensembled TF/Keras versus PyTorch scores, and each versus the officially published PromoterAI scores. r = 1.000, MAE = 0.000 in every panel.

Benchmark equivalence. Scoring the public benchmark variant sets released alongside the paper — `CAGI5_saturation`, `GEL_RNA`, `GTEx_eQTL`, `GTEx_outlier`, `MPRA_eQTL`, `MPRA_saturation`, and `UKBB_proteome` (under/over/null variant categories per dataset) — with the torch checkpoints reproduces the TF/Keras ensemble's under-vs-over, under-vs-null, and over-vs-null AUROCs to within ~1e-6:

Dataset	***n* (under/over/null)**	**under-vs-over**	**under-vs-null**	**over-vs-null**
CAGI5_saturation	976 / 499 / 5,095	0.8845	0.7939	0.7153
GEL_RNA	309 / 239 / 609	0.9002	0.7757	0.7802
GTEx_eQTL	191 / 218 / 393	0.8697	0.7876	0.7503
GTEx_outlier	206 / 161 / 382	0.8938	0.7972	0.7423
MPRA_eQTL	70 / 74 / 542	0.9004	0.8069	0.8278
MPRA_saturation	773 / 275 / 3,981	0.8707	0.8675	0.7010
UKBB_proteome	182 / 69 / 760	0.9116	0.7718	0.7757

(Torch AUROCs shown; the matching TF/Keras run agrees on every value to at least five decimal places.) The per-dataset and aggregate ensemble variant scores underlying these AUROCs also match nearly exactly between the two implementations (Pearson r = 1.0000 for each of the seven datasets and for all 16,004 variants combined):



PyTorch versus TF/Keras ensemble variant scores on each of the paper's released benchmark datasets (Pearson r = 1.0000 in every panel) and combined across all 16,004 variants (bottom centre).

Training equivalence. Inference equivalence doesn't guarantee the training loop itself matches — a converter can produce an identical model while the from-scratch training and fine-tuning code silently diverges from Illumina's Keras implementation. `train.py` and `finetune.py` clip gradients per parameter (matching Keras' `clipnorm` semantics, as opposed to a single norm across all parameters jointly), set `BatchNorm`'s momentum to the value

equivalent to Keras' `momentum=0.99`, and count `steps_per_epoch` the same way Keras does. Multi-species training also matches Keras' handling of a batch's inactive species: its loss term stays in the graph as a zero-weighted zero rather than being dropped, so weight decay still applies to every head on every step as it does in Keras, and each batch is drawn from a single species rather than mixed across species. A cross-framework test suite runs one training step through numerically identical converted weights in both frameworks — at toy scale, and, for all four released checkpoints, at the real published scale (`num_blocks=24`, `model_dim=1024`) on GPU against Illumina's own SavedModels — and checks agreement on the loss, gradients, AdamW parameter deltas, and BatchNorm running-stat updates via a per-tensor cosine-similarity/relative-L2 pass rate rather than strict elementwise tolerances. All four real-checkpoint configurations pass; for the base (`hg38`, `hg38_mm10`) checkpoints, forward-pass prediction agreement is cosine = 1.0000 with relative L2 under 0.5%. The one remaining, characterized divergence is framework-inherent rather than a porting gap: Keras' `AdamW` places its `epsilon` term differently than PyTorch's, transiently damping its first ~1,000 optimizer steps more strongly even with a matching `epsilon`. See `notes/implementation.md` and `docs/training.md` in the repo for the full derivation.

What Can You Do With This?

Beyond variant scoring, `load_pretrained()` exposes the full model for anything you'd normally do with a PyTorch sequence model — track prediction, embeddings, and DeepLIFT/SHAP attribution [3]:

```
import torch
import torch.nn as nn
from tangermeme.deep_lift_shap import deep_lift_shap
from promoterai_torch.dataset import onehot_encode
from promoterai_torch.utils import load_pretrained

model, args = load_pretrained("models/promoterAI_v1_hg38_mm10_finetune.pt")
model.eval()

seq = "ACGT" * (args["input_length"] // 4)          # replace with your sequence
x = torch.from_numpy(onehot_encode(seq)).unsqueeze(0)  # (1, L, 4)

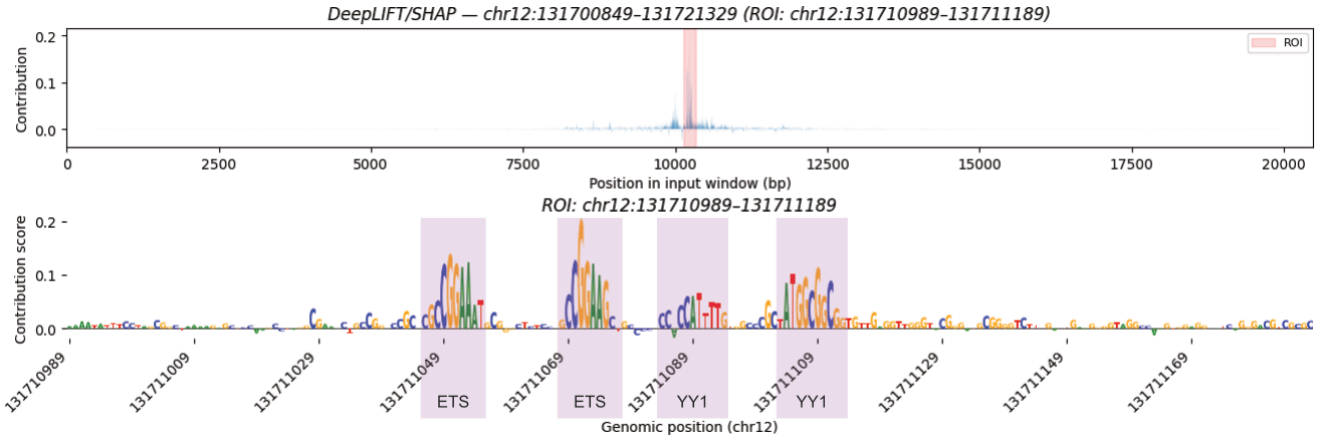
with torch.no_grad():
    predictions = model(x)          # tuple of (1, output_length, n_tracks) per species head
    embeddings = model.encode(x)    # (1, input_length, model_dim), final MetaFormer block
output

# DeepLIFT/SHAP via tangermeme: every non-linearity is a distinct, named nn.ReLU()
# instance, which is exactly what tangermeme's deep_lift_shap requires.
class PromoterAIWrapper(nn.Module):
    def __init__(self, model):
        super().__init__()
        self.model = model

    def forward(self, x):                           # x: (B, 4, L) channels-first
        out = self.model(x.transpose(1, 2))         # PromoterAI expects (B, L, 4)
        return out[0].mean(dim=(1, 2)).unsqueeze(1)  # (B, 1) — mean over positions and tracks

wrapper = PromoterAIWrapper(model)
x_chfirst = x.transpose(1, 2)  # (1, 4, input_length), channels-first for tangermeme
attributions = deep_lift_shap(wrapper, x_chfirst, n_shuffles=20, device="cuda", batch_size=1)
# attributions: (1, 4, input_length) — per-position, per-base importance
```

Track prediction returns per-position predictions for all 498 human tracks the model was trained on (histone marks, TF ChIP-seq, ATAC-seq, RNA-seq), plus the 472-track mouse head; embeddings are the per-position hidden state after the final MetaFormer block, `(B, L, model_dim)`. The `deep_lift_shap` call above (transposing to channels-first, reducing the output heads to a scalar via the wrapper) is what produces per-base attribution maps like the one below:



Per-base DeepLIFT/SHAP contribution scores at the SFSWAP promoter (chr12:131,700,849–131,721,329), zoomed into the 200 bp region of interest (chr12:131,710,989–131,711,189).

Fair warning on cost: DeepLIFT/SHAP on this model is not cheap — at TF32 with `n_shuffles=20` and `batch_size=1`, expect ~92s and ~71GB of VRAM per sequence on an A100 80GB. For an order-of-magnitude anchor (not a like-for-like benchmark — different attribution method, hardware, and input-window size), the Cherimoya post [12] measured *in silico* mutagenesis on a 1 kb locus at 0.08s (Cherimoya), 0.38s (ChromBPNet), 1.24s (AlphaGenome 2kb), 64.5s (Borzoi), and 324.3s (AlphaGenome) on an H200 GPU (bf16, `torch.compile`); peak batch-1 VRAM for a full-sequence forward pass on the same hardware was 0.14GB, 0.19GB, 17.9GB, and 117GB for Cherimoya, ChromBPNet, Borzoi, and AlphaGenome (1Mb) respectively.

Training and Fine-Tuning

The repo also includes the full training pipeline, not just inference: HDF5 preprocessing of track and sequence data per chromosome, multi-GPU training via `torchrun`, checkpoint/resume handling, and a fine-tuning script that trains only the first output head on a variant set (matching PromoterAI's own fine-tuning protocol on GTEx outlier data) while keeping the rest of the backbone — including BatchNorm statistics — frozen in inference mode.

```
promoterai-torch train \
    --checkpoint_folder checkpoints/run1 \
    --hdf5_human_folder data/hdf5/human \
    --input_length 20480 --output_length 4096 \
    --num_blocks 24 --model_dim 1024 --batch_size 32
```

This hasn't been used to reproduce Illumina's exact published model from scratch — that would require their full training corpus — but it has been verified to run end-to-end and to match the original's documented training/fine-tuning behavior wherever that behavior is checkable.

Code and Tutorials

- Repository: github.com/genomicsxai/promoterai-torch
- PyPI: `promoterai-torch`
- Worked examples (paper benchmark reproduction, track-level parity checks, TERT/SFSWAP/DNAJC9 notebooks): `examples/`

License

`promoterai-torch` is an independent reimplementation, not an official Illumina product or publication. Its PyTorch code was written from PromoterAI's published architecture description and released hyperparameters — the depth, width, dilation schedule, output-head structure, and naming scheme needed for the two implementations to be numerically compatible — rather than by transliterating Illumina's TensorFlow/Keras source. `promoterai-torch`'s own code is MIT-licensed.

The original PromoterAI codebase is released under the PolyForm Strict License 1.0.0, which permits noncommercial, research, and educational use but withholds the right to distribute the software or "changes or new works based on" it. Illumina's pretrained weights and precomputed variant scores are gated separately, under Illumina's own academic-only data license — see Illumina/PromoterAI for the license agreement and commercial-licensing contact (`AI_licensing@illumina.com`). This repo contains no Illumina code, models, or scores; if you convert and use the original weights yourself, you are responsible for complying with Illumina's terms. Converted checkpoints should not be redistributed.

Acknowledgements

This work builds directly on the architecture and training protocol described by Illumina's PromoterAI team, and on `tangermeme` for attribution tooling.

References

1. Jaganathan, K., Ersaro, N., Novakovsky, G. et al. Predicting expression-altering promoter mutations with deep learning. *Science* 389, eads7373 (2025). https://doi.org/10.1126/science.ads7373
2. Illumina/PromoterAI (official TensorFlow implementation). https://github.com/Illumina/PromoterAI
3. Schreiber, J. tangermeme: A toolkit for understanding cis-regulatory logic using deep learning models. *bioRxiv* (2025). https://www.biorxiv.org/content/10.1101/2025.08.08.669296v2
4. Huang, F. W., Hodis, E., Xu, M. J., Kryukov, G. V., Chin, L. & Garraway, L. A. Highly recurrent TERT promoter mutations in human melanoma. *Science* 339, 957–959 (2013). https://doi.org/10.1126/science.1229259
5. Horn, S., Figl, A., Rachakonda, P. S. et al. TERT promoter mutations in familial and sporadic melanoma. *Science* 339, 959–961 (2013). https://doi.org/10.1126/science.1230062
6. Killela, P. J., Reitman, Z. J., Jiao, Y. et al. TERT promoter mutations occur frequently in gliomas and a subset of tumors derived from cells with low rates of self-renewal. *Proc Natl Acad Sci USA* 110, 6021–6026 (2013). https://doi.org/10.1073/pnas.1303607110
7. Bell, R. J. A., Rube, H. T., Xavier-Magalhães, A. et al. Understanding TERT Promoter Mutations: A Common Path to Immortality. *Mol Cancer Res* 14, 315–323 (2016). https://doi.org/10.1158/1541-7786.MCR-16-0003
8. Schreiber, J., Lorbeer, F. K., Heinzl, M., Reiter, F., Rafanel, B., Lu, Y., Stark, A. & Noble, W. S. Programmatic design and editing of cis-regulatory elements. *bioRxiv* (2025). https://doi.org/10.1101/2025.04.22.650035

9. Avsec, Ž., Agarwal, V., Visentin, D. et al. Effective gene expression prediction from sequence by integrating long-range interactions. *Nat Methods* 18, 1196–1203 (2021). https://doi.org/10.1038/s41592-021-01252-x
10. Hingerl, J. C., Karollus, A. & Gagneur, J. Flashzoi: an enhanced Borzoi for accelerated genomic analysis. *Bioinformatics* 41, btaf467 (2025). https://doi.org/10.1093/bioinformatics/btaf467
11. Bredikhin, D., Buendia, A., Kjellberg, M., Zou, C., Tu, X. & Kundaje, A. Porting AlphaGenome to PyTorch. *Genomics x AI Blog* (2026). https://genomicsxai.github.io/blogs/2026-004/
12. Ramirez, C., Aruva, A. M., Weng, Z. & Schreiber, J. Cherimoya: a lightweight genomic sequence-to-function model built for large-scale design and understanding. *Genomics x AI Blog* (2026). https://genomicsxai.github.io/blogs/2026-011/